

高并发库存扣减为什么需要令牌限流？

以 12306 为背景，如何防止大量的用户请求无意义访问，因为票就那些，只有少数人能买到。为此，我们通过令牌来完成防止用户无意义访问以及库存扣减功能。

很多同学不理解通过余票缓存也可以进行限流，为什么要有令牌容器这个东西？这篇文章和大家说下。

业务背景

购买票扣减关系

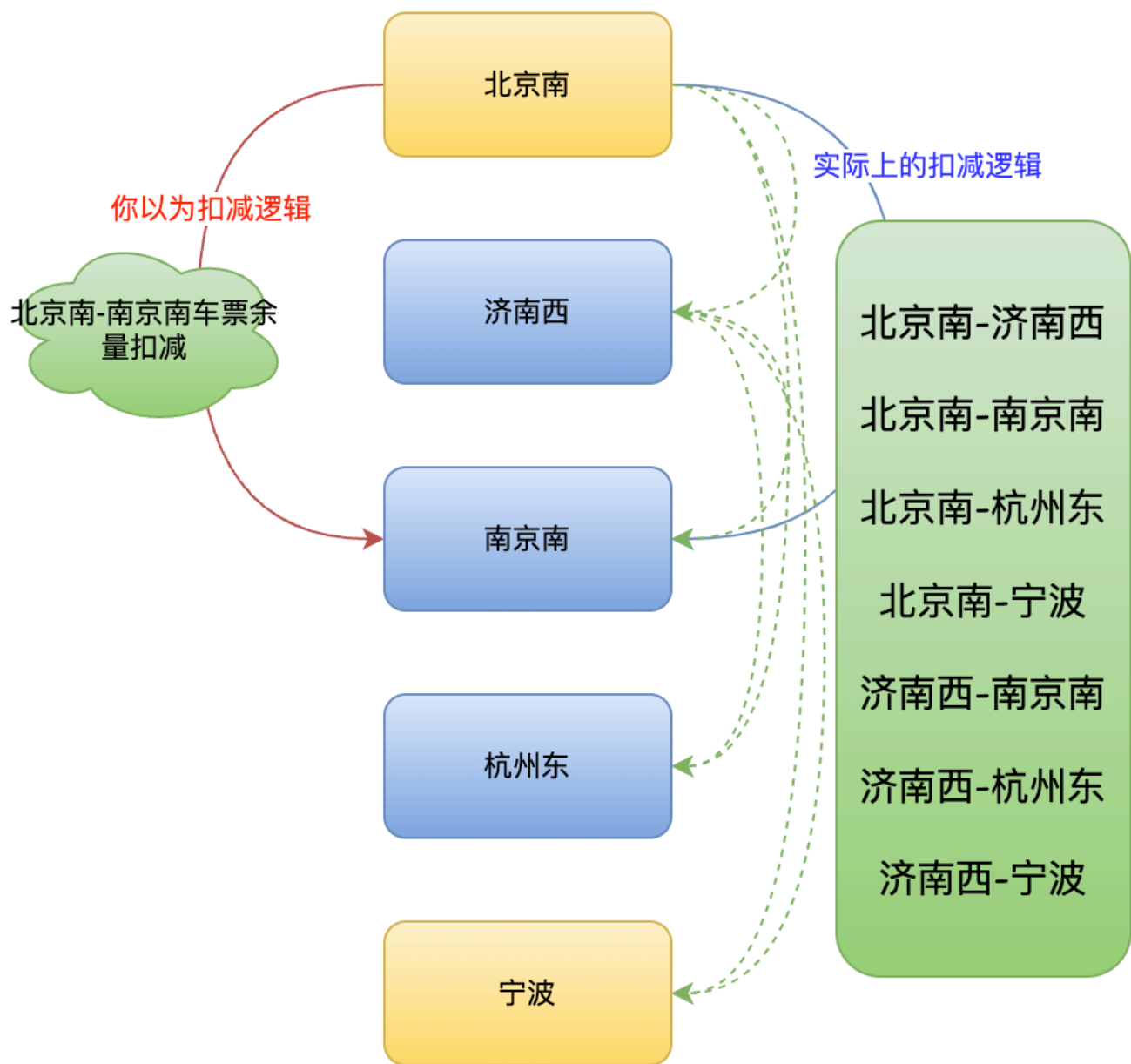
以 12306 的 G35 列车举例。

北京 → 杭州 （08月20日 周天） 共计4个车次

车次	出发站	到达站	出发时间	到达时间	历时	商务座 特等座	一等座	二等座 二等包座
+ G35	北京南	杭州东	09:56	15:14	5:18			
+ G39	北京南	杭州东	09:56	15:14	5:18			
+ D717	北京南	杭州东	09:56	15:14	5:18			
+ Z284	北京南	杭州东	09:56	15:14	5:18			

如果查询结果中显示有卧铺票价的，均为上铺票价，供您参考。具体票价以您确认支付时实际购买的铺别票价为准。如

假如我要买北京南到南京南，扣减逻辑如下：



数据库存储列车座位记录

Items

Queries

History

Search for item...

Functions

Tables

t_carriage

t_region

t_seat

t_station

t_ticket

t_train

t_train_station

t_train_station_price

t_train_station_relation

t_order_0

t_order_1

t_order_10

t_order_11

t_order_12

t_order_13

t_order_14

t_order_15

t_order_2

t_order_3

t_order_4

t_order_5

t_order_6

t_order_7

t_order_8

t_order_9

t_order_item_0

t_order_item_1

Raw SQL

Pattern

Applied

Export

SQL

Show: 30F

Insert: 30I

Remove: 30O

Apply All: 30Return

Up: 30↑

Down: 30↓

Columns: 30←

Unset

Apply All

id	train_id	carriage_number	seat_number	seat_type	start_station	end_station	price	seat_status	creat
1683022024079286272	3	01	01A	0	北京	德州	7500	0	2023-07-23
1683022024129617920	3	01	01D		0 北京	德州	7500	0	2023-07-23
1683022024158978048	3	01	01F		0 北京	德州	7500	0	2023-07-23
1683022024184143872	3	01	02A		0 北京	德州	7500	0	2023-07-23
1683022024209309696	3	01	02C		0 北京	德州	7500	0	2023-07-23
1683022024234475520	3	01	02D		0 北京	德州	7500	0	2023-07-23
1683022024259641344	3	01	02F		0 北京	德州	7500	0	2023-07-23
1683022024284807168	3	01	03A		0 北京	德州	7500	0	2023-07-23
1683022024301584384	3	01	03C		0 北京	德州	7500	0	2023-07-23
1683022024314167296	3	01	03D		0 北京	德州	7500	0	2023-07-23
1683022024330944512	3	01	03F		0 北京	德州	7500	0	2023-07-23
1683022024343527424	3	01	04A		0 北京	德州	7500	0	2023-07-23
1683022024356110336	3	01	04C		0 北京	德州	7500	0	2023-07-23
1683022024372887552	3	01	04D		0 北京	德州	7500	0	2023-07-23
1683022024381276160	3	01	04F		0 北京	德州	7500	0	2023-07-23
1683022024398053376	3	01	05A		0 北京	德州	7500	0	2023-07-23
1683022024410636288	3	01	05C		0 北京	德州	7500	0	2023-07-23
1683022024423219200	3	01	05D		0 北京	德州	7500	0	2023-07-23
1683022024435802112	3	01	05F		0 北京	德州	7500	0	2023-07-23
1683022024448385024	3	01	06A		0 北京	德州	7500	0	2023-07-23
1683022024460967936	3	01	06C		0 北京	德州	7500	0	2023-07-23
1683022024473550848	3	01	06D		0 北京	德州	7500	0	2023-07-23
1683022024490328064	3	01	06F		0 北京	德州	7500	0	2023-07-23

Details

Assistant

Search for field...

id

longlong

1683022024079286272

train_id

longlong

3

carriage_number

var_string

01

车厢

seat_number

var_string

01A

座位编号

seat_type

long

0

start_station

var_string

北京

起始站点

end_station

var_string

德州

到达站点

price

long

7500

seat_status

long

0

create_time

datetime

2023-07-23 15:51:34

update_time

datetime

2023-07-23 15:51:34

del_flag

tiny

0

座位类型：高铁上午、一、二等座

座位状态 0: 可售 1: 锁定 2: 已售 4

技术实现方案

Redis 缓存车票库存预先扣减

将列车余票放到缓存中，充当限流作用，通过 LUA 脚本扣减余票后，扣减成功代表余票数量充足，才可以进入流程分配座位，并创建订单，扣减失败请求打回。

因为扣减列车站点余票涉及到比较多的扣减关系，所以这里要使用 LUA 脚本。



构建库存令牌容器，拆分开限流和库存

相对于上一种方案，拆分了一个令牌容器，其中存放的令牌就是列车余票数据。如果用户购买车票，需要先扣减令牌容器，扣减成功代表余票数量充足，扣减失败请求打回。

用户查询列表所看到的余票数量，通过余票缓存，而不是令牌容器。

扣减令牌容器后，对余票缓存并没有影响，只有在修改了座位表后，才会通过 Canal 形式监听并减少余票缓存。



为什么要拆分令牌容器？

因为有个极端情况，比如说，扣减余票缓存容器成功后，然后应用宕机，导致余票缓存和余票数据库库存不一致。

如果有令牌容器，发现没有剩余可用令牌，那么我们可以进行兜底判断，也可以叫做二次检查。触发一个请求去比对数据库是否还有值，如果有的话，那么就把令牌容器缓存删除，下个用户再购票时，重新加载即可。

```
// 设置这个对象的关键在于，避免大量的用户同一时间访问刷新 Token
private final Cache<String, Boolean> tokenTicketsRefreshMap = Caffeine.newBuilder()
    .expireAfterWrite(10, TimeUnit.MINUTES)
    .build();

@Override
public TicketPurchaseRespDTO purchaseTicketsV2(PurchaseTicketReqDTO requestParam) {
    // 责任链模式，验证 1：参数必填 2：参数正确性 3：乘客是否已买当前车次等...

    purchaseTicketAbstractChainContext.handler(TicketChainMarkEnum.TRAIN_PURCHASE_TICKET_FILTER.name(), requestParam);
    // 获取令牌结果
    TokenResultDTO tokenResult =
ticketAvailabilityTokenBucket.takeTokenFromBucket(requestParam);
    // 获取令牌错误
    if (tokenResult.getTokenIsNull()) {
        // 原子操作：若 Key 不存在，插入 true 并返回 null；若存在，直接返回当前值
        Boolean isRefreshing =
tokenTicketsRefreshMap.asMap().putIfAbsent(requestParam.getTrainId(), true);
        if (isRefreshing == null) {
            tokenIsNullRefreshToken(requestParam, tokenResult);
        }
        throw new ServiceException("列车站点已无余票");
    }
    // .....
}
```

如果用户返回 Token 为空的话则进行刷新，方法如下：

```
private final ScheduledExecutorService tokenIsNullRefreshExecutor =
Executors.newScheduledThreadPool(1);

private void tokenIsNullRefreshToken(PurchaseTicketReqDTO requestParam, TokenResultDTO
tokenResult) {
    RLock lock = redissonClient.getLock(String.format(LOCK_TOKEN_BUCKET_ISNULL,
requestParam.getTrainId()));
    // 尝试获取分布式锁，如果不成功则直接返回即可
    if (!lock.tryLock()) {
        return;
    }
    // 为什么要延迟更新？因为如果不延时，一辆列车突发将列车票售空，可能数据库还没有扣减完，所
以有个 10 秒缓冲时间
    tokenIsNullRefreshExecutor.schedule(() -> {
        try {
            // 组装出座位类型以及每个座位类型下购票人数
            List<Integer> seatTypes = new ArrayList<>();
            Map<Integer, Integer> tokenCountMap = new HashMap<>();
            tokenResult.getTokenIsNullStations().stream()
                .map(each -> each.split("_"))
                .forEach(split -> {
                    int seatType = Integer.parseInt(split[0]);
                    seatTypes.add(seatType);
                    tokenCountMap.put(seatType, Integer.parseInt(split[1]));
                });
            // 获取数据库中座位类型对应的余票数量
            List<SeatTypeCountDTO> seatTypeCountDTOList =
seatService.listSeatTypeCount(Long.parseLong(requestParam.getTrainId()),
requestParam.getDeparture(), requestParam.getArrival(), seatTypes);
            for (SeatTypeCountDTO each : seatTypeCountDTOList) {
                Integer tokenCount = tokenCountMap.get(each.getSeatType());
                // 如果判断数据库余票数大于扣减不成功的数量
                if (tokenCount <= each.getSeatCount()) {
                    ticketAvailabilityTokenBucket.delTokenInBucket(requestParam);
                    break;
                }
            }
        } finally {
            lock.unlock();
        }
    });
}
```

```
    }, 10, TimeUnit.SECONDS);  
}
```

通过以上方式，非常好解决了数据库存在可购买余票，而缓存中限流时无法购买的情况，可以保证列车余票最终一定会被售卖。

常见问题答疑

可否仅使用余票缓存做余票数量展示和限流效果？

这个是可以的，在大多数情况下，因为咱们技术方案比较复杂，有些面试官很难理解这个令牌限流。所以，我们可以使用传统秒杀架构，余票缓存直接扣减，然后再扣减数据库，就完成了所有流程。

传统秒杀架构可能存在什么问题？

<https://t.zsxq.com/9i0NM>

更新: 2026-03-11 20:09:48

原文: <<https://www.yuque.com/magestack/12306/gdqnbhp10xx9g9kv>>